Lesson 11 · Security & Identity with Entra ID

Authentication schemes, authorization policies, JWT validation, Entra ID and the OAuth flows, a YARP gateway, secrets and hardening — proven by a 401/403/200 test matrix

Contents

1 · Why this lesson — the security you have shipped, in .NET terms 1
1.1 · The numbers to leave this lesson with 2
2 · The model — schemes, policies and the pipeline 2
2.1 · Concept map — Spring Security and NestJS → ASP.NET Core 2
2.2 · The pipeline of L11.SecureQuoteApi 3
2.3 · Where a 401, a 403 and a 429 come from 4
2.4 · The same valid token, two middleware orders 4
2.5 · An endpoint nobody protected, with and without a fallback policy 5
3 · Tokens — what JwtBearer checks, and what it renames 5
3.1 · Eight tokens against one set of parameters 6
3.2 · What the handler accepted 6
3.3 · How long after exp a token is still accepted 6
3.4 · Reading scp — the same helper in C# and in Visual Basic 7
3.5 · Validating in Visual Basic, mapping on and off 7
3.6 · One underwriter token, read twice 7
3.7 · Delegated tokens and app-only tokens 8
4 · Entra ID, External ID and the other authorities 8
4.1 · The same API behind five authorities 8
4.2 · Any OpenID Connect authority 9
4.3 · Entra ID through Microsoft.Identity.Web 9
5 · Flows — who gets a token for whom 9
5.1 · Six flows and credentials for the MotorQuote platform 10
5.2 · A customer quote request that calls a rating partner 11
5.3 · Calling the partner as the user, and as the API 12
5.4 · A federated credential is a file path 12
6 · Authorization design — policies, resources and the gateway 12
6.1 · Access rules — Spring Security paths vs ASP.NET Core named policies 13
6.2 · Policy first, then the resource check 13
6.3 · Who may read or accept this quote 13
6.4 · The YARP gateway: coarse checks per route 14
6.5 · Routes — Spring Cloud Gateway vs YARP 14
7 · Secrets and hardening — STRIDE for one API 15
7.1 · OWASP Top 10:2025 — what the lesson 11 samples cover 15
7.2 · STRIDE for the quote API — threat, control, evidence 15
7.3 · CORS and a per-caller rate limit 16
7.4 · Five ways to read one protected share link 16
7.5 · What Data Protection returned 17
8 · VB and legacy — Forms and Windows authentication 17
8.1 · .NET Framework security pieces and their .NET 10 replacements 17
8.2 · Choosing the replacement for a legacy login 18
8.3 · A Forms-era principal read as claims, in Visual Basic 18
8.4 · The legacy role, under the long claim type 18
9 · Hands-on — test-signed tokens and a 401/403/200 matrix 18
9.1 · Minting test tokens 19
9.2 · Pointing both apps at the test key 19
9.3 · The expected status of every caller 20
9.4 · Commands 20
9.5 · What you should see 20
9.6 · Test cases per test class 20
9.7 · Lines of code per project 20
9.8 · The status matrix as a heat map 21
9.9 · Outcomes across the matrix 21

Page numbers are resolved from the rendered PDF. C# and VB panels are cut from the compiled samples; outputs and counts marked measured come from real runs of those samples.

1 · Why this lesson — the security you have shipped, in .NET terms

You have secured this kind of API before, and most of the ideas carry over; what changes is where .NET puts each decision and which defaults it chooses for you. You have built Spring Security resource servers, NestJS guards and gateway-level authorization in Zuul, and integrated single sign-on with Entra ID, Azure AD B2C, Auth0 and Cognito. ASP.NET Core has the same parts: an authentication *scheme* turns a request into a `ClaimsPrincipal`, an authorization *policy* decides, and both are wired in `Program.cs` in the order you write them.

The defaults are where an experienced engineer from another stack gets hurt. A .NET API accepts a token up to 5 minutes after it expired, where Spring allows 60 seconds; it renames `scp` and `roles` to long URIs unless you switch mapping off; and an endpoint with no authorization requirement is public unless you set a fallback policy. Each is a one-line fix, and each is

measured by a sample in this lesson.

The identity landscape moved while you were shipping. Azure AD B2C has not been sold to new customers since 1 May 2025 and is supported until at least May 2030; Microsoft Entra External ID is the customer-identity product now (External ID FAQ) • Verified. Workload identity federation lets a GitHub Actions job, a Kubernetes pod — EKS included — or an AWS workload obtain an Entra token without a client secret (Workload identity federation).

Every claim here is backed by code that runs. A MotorQuote API, a YARP gateway, 49 tests including a 36-case status matrix, a token lab, a Data Protection lab and a Visual Basic console that reads the same claims. The tests sign their own tokens, so none of it needs a tenant or a network.

1.1 · The numbers to leave this lesson with

DEFAULT CLOCK SKEW	AZURE AD B2C	STATUS MATRIX	LESSON 11 TESTS	LESSON 11 SAMPLES
5 min	**closed**	**36 cases**	**49**	**784 lines**
Spring Security: 60 s · measured + verified	to new customers since 1 May 2025 · verified	9 callers × 4 endpoints · measured	all passing, no tenant needed · measured	C# 705 · VB 79 · 6 projects · measured

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured architecture cloud

i Before you start

1. **You need lesson 08 (ASP.NET Core Web APIs)** — hosting, dependency injection, options, minimal APIs and middleware — and the `WebApplicationFactory` basics of lesson 10 (Testing, TDD & 100% Coverage). Async code follows lesson 05 (Async, Tasks & Concurrency).
2. **You will build six projects:** `L11.SecureQuoteApi` (web API: JwtBearer or Microsoft.Identity.Web, policies, a resource handler, hardening), `L11.Gateway` (YARP), `L11.SecureQuoteApi.Tests` (xUnit), `L11.TokenLab` and `L11.ProtectLab` (C# consoles) and `L11.ClaimsVb` (a Visual Basic console).
3. **No Entra tenant is needed.** Tests sign tokens with an RSA key created for each test run. The Entra-specific registration and the downstream partner calls compile, and one test checks the registration's options, but nothing contacts Microsoft.
4. Tenant and client IDs are zero-filled placeholders and every host except Entra's public sign-in address ends in `example.test`. Premiums are **illustrative, not a real tariff**: the quote API prices with the MotorQuote tariff every lesson shares — Class 1 at 2.1% of the sum insured, 0.4% stamp duty, 7% VAT — reduced to what a bare sum insured allows (no loadings, no no-claim discount, because the request carries no driver or claims data).

2 · The model — schemes, policies and the pipeline

ASP.NET Core splits security into two services: an authentication scheme says who the caller is, an authorization policy says what they may do. `AddAuthentication().AddJwtBearer()` registers a scheme; `UseAuthentication()` runs it for each request and stores the result in `HttpContext.User`. That is Spring's `SecurityContext`, but carried by the request rather than a thread: `ClaimsPrincipal.Current` and `Thread.CurrentPrincipal` are not set by default (Migrate from static ClaimsPrincipal access). Policies are named requirements registered once and attached to endpoints with `RequireAuthorization("name")` or `[Authorize(Policy = …)]`.

A 401 and a 403 come from different places. Authorization evaluates the endpoint's policy first. When the policy is not satisfied it looks at the caller: no authenticated user means the scheme's *challenge*, a 401 with `WWW-Authenticate: Bearer`; an authenticated user gets *forbid*, a 403. A check about one particular quote can only run inside the endpoint, so its 403 comes from there (diagram 2.3).

The pipeline is code, so its order is your responsibility. CORS runs before authentication because a preflight request carries no token, and authentication runs before authorization (Middleware order). Swap the last two and the app compiles, starts and answers a valid token with 401 — test 2.4 proves it. If you call neither method and the services are registered, `WebApplication` adds both right after routing.

Nothing is protected until you say so. An endpoint without authorization metadata is public. A fallback policy applies to every endpoint that names no policy of its own, which turns "forgot the attribute" from a data leak into a 401 (Require authenticated users). Test 2.5 shows both answers for one forgotten endpoint.

2.1 · Concept map — Spring Security and NestJS → ASP.NET Core

You already know	In .NET	Kind	Watch for
`SecurityFilterChain` bean	`UseAuthentication()` + `UseAuthorization()` in `Program.cs`	• Trap	Order is statement order; the wrong order still compiles (2.4)
`SecurityContextHolder`	`HttpContext.User` — a `ClaimsPrincipal`, or a `ClaimsPrincipal` handler parameter	• Renamed	No static access by default — pass the principal into services
`oauth2ResourceServer { jwt { } }`	`AddJwtBearer(o => o.Authority = …)`	• Same idea	Discovery document and JWKS keys are fetched and cached for you
`jwt.audiences` (off unless set)	`Audience` / `ValidAudience`	• Trap	.NET fails closed: with no audience configured every token is rejected (3.2)
`JwtTimestampValidator`, 60 s skew	`TokenValidationParameters.ClockSkew`, 5 min	• Trap	Set it: an expired token passed the gateway until it was lowered (3.3)
`SCOPE_` authorities from `scp`	The raw `scp` claim — after `MapInboundClaims = false`	• Trap	Default mapping renames claims; one string holds all scopes (3.4)
`hasAuthority("SCOPE_x")`	Policy: `RequireAssertion(c => c.User.HasScope(x))`	• Renamed	`RequireClaim("scp", x)` compares the whole space-separated string
`@PreAuthorize("hasRole('X')")`	`.RequireAuthorization("policy")` or `[Authorize(Policy = …)]`	• Renamed	Policies are named once, attached per endpoint
`PermissionEvaluator` / `@PostAuthorize`	`IAuthorizationService.AuthorizeAsync(user, quote, op)` + a handler	• Different	Imperative: called inside the endpoint, after the quote is loaded (6.3)
`anyRequest().authenticated()`	`SetFallbackPolicy(RequireAuthenticatedUser)`	• Trap	Not the default: without it, a new endpoint with no metadata is public (2.5)
`AuthenticationEntryPoint` / `AccessDeniedHandler`	Scheme challenge → 401 · forbid → 403	• Renamed	`Results.Forbid()` inside an endpoint produces the 403
NestJS `AuthGuard('jwt')` + passport-jwt	JwtBearer scheme + endpoint policy	• Same idea	Guards map to policies; interceptors to middleware or endpoint filters
`@WithMockUser`, MockMvc `jwt()`	Test-signed JWT + `WebApplicationFactory`	• Different	Real signature, issuer, audience and lifetime checks run in the tests (9)
CSRF filter on by default	Antiforgery for form posts and cookie auth only	• Different	Bearer tokens in a header are not sent automatically, so they are not a CSRF target (Prevent cross-site request forgery)

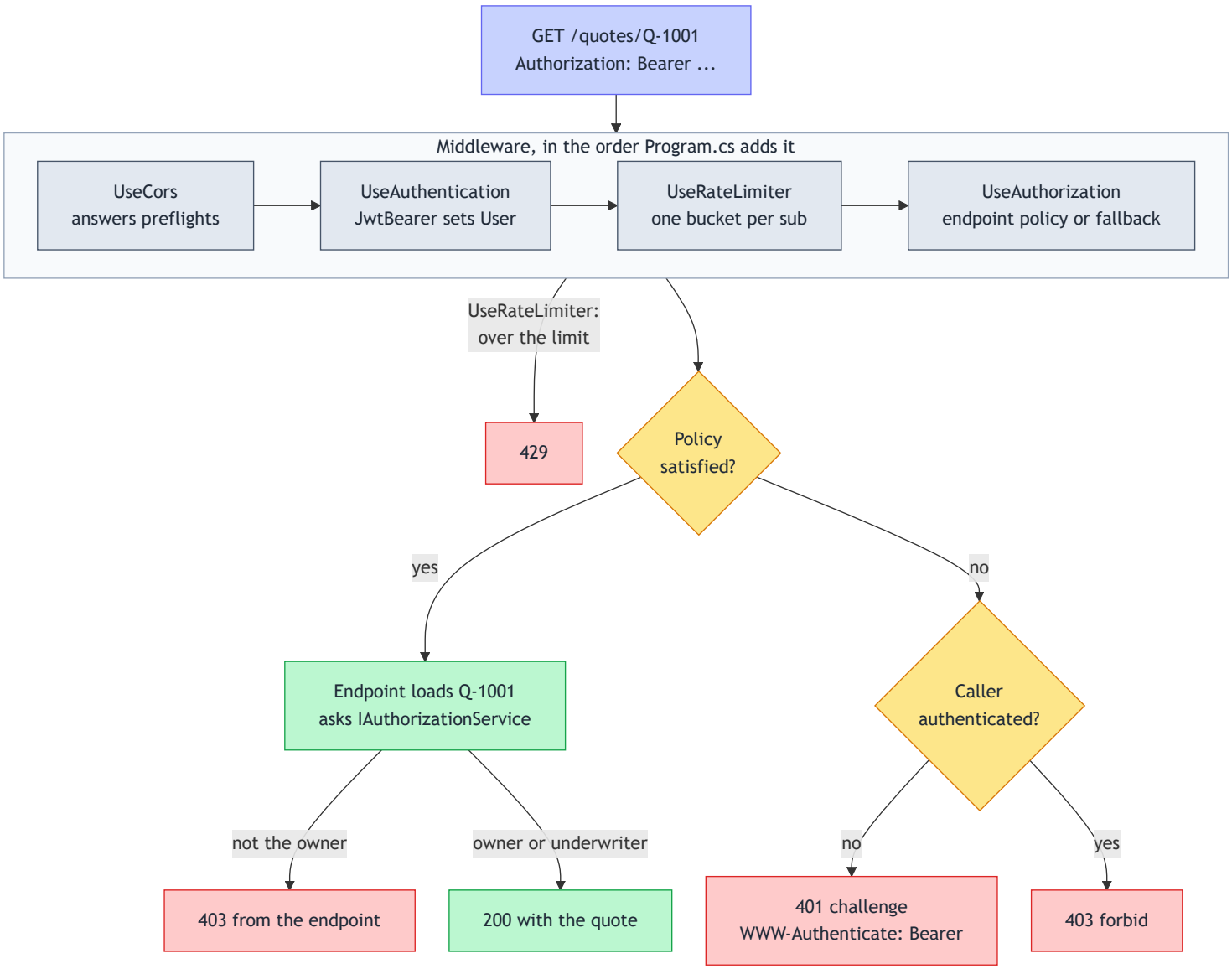
2.2 · The pipeline of L11.SecureQuoteApi

C# — samples/L11.SecureQuoteApi/Program.cs · #region pipeline

```
// No UseHttpsRedirection or UseHsts: an API listens on HTTPS only. A redirect
// arrives after the client has already sent its bearer token over plain HTTP.
app.UseForwardedHeaders();      // first: the client IP the limiter keys on
app.UseCors();                  // before auth: a CORS preflight carries no token
app.UseAuthentication();        // who are you?  -> HttpContext.User
app.UseRateLimiter();           // needs the subject, so after authentication
app.UseAuthorization();         // may you?      -> 401, 403 or the endpoint
```

> **Read it top to bottom as the order every request takes.** The rate limiter sits between authentication and authorization because it partitions by the token's `sub`, which only exists after `UseAuthentication`. The first comment is a design decision explained in section 7: an API does not redirect HTTP to HTTPS. `UseForwardedHeaders` is first so that every later step sees the caller's address, not the load balancer's (7.3).

2.3 · Where a 401, a 403 and a 429 come from



indigo = request · slate = the security middleware in Program.cs order, inside the grey box (UseForwardedHeaders runs before it, 7.3) · amber = decision · green = the endpoint runs · red = rejected · the 429 leaves the rate limiter

2.4 · The same valid token, two middleware orders

C# — samples/L11.SecureQuoteApi.Tests/MiddlewareOrderTests.cs · #region order-trap

```
[Theory]
[InlineData(true, HttpStatusCode.OK)]
[InlineData(false, HttpStatusCode.Unauthorized)]   // the SAME valid token
public async Task Authentication_must_run_before_authorization(
    bool authenticateFirst, HttpStatusCode expected)
{
    var builder = WebApplication.CreateSlimBuilder();
    builder.WebHost.UseTestServer();
    builder.Services.AddAuthorization();
    builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(o =>
        {
            o.Audience = TestTokens.Audience;
            o.Configuration = new OpenIdConnectConfiguration { Issuer = TestTokens.Issuer };
            o.Configuration.SigningKeys.Add(TestTokens.Key);
        });
    await using var app = builder.Build();

    if (authenticateFirst) { app.UseAuthentication(); app.UseAuthorization(); }
    else { app.UseAuthorization(); app.UseAuthentication(); }   // compiles, runs, rejects
    app.MapGet("/quotes/{id}", (string id) => id).RequireAuthorization();
    await app.StartAsync();
```

```
    using var client = app.GetTestClient();
    client.DefaultRequestHeaders.Authorization =
        new AuthenticationHeaderValue("Bearer", TestTokens.Create("cust-a", "Quotes.Read"));
    using var response = await client.GetAsync("/quotes/Q-1001");

    Assert.Equal(expected, response.StatusCode);
}
```

Both cases pass: the order that runs authorization first returns 401 for a token that the other order accepts. `UseAuthorization` asks whether `HttpContext.User` is authenticated; nothing has authenticated it yet, so it challenges. There is no warning at build or start-up — a test like this one is the guard.

2.5 · An endpoint nobody protected, with and without a fallback policy

C# — samples/L11.SecureQuoteApi.Tests/MiddlewareOrderTests.cs · #region fallback-trap

```
[Theory]
[InlineData(false, HttpStatusCode.OK)]             // "forgot the attribute": public
[InlineData(true, HttpStatusCode.Unauthorized)]    // fallback policy: needs a user
public async Task Endpoint_without_metadata_is_public_unless_a_fallback_policy_exists(
    bool fallbackPolicy, HttpStatusCode expected)
{
    var builder = WebApplication.CreateSlimBuilder();
    builder.WebHost.UseTestServer();
    builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer();
    var authorization = builder.Services.AddAuthorizationBuilder();
    if (fallbackPolicy)
    {
        authorization.SetFallbackPolicy(new AuthorizationPolicyBuilder()
            .RequireAuthenticatedUser().Build());
    }
    await using var app = builder.Build();
    app.UseAuthentication();
    app.UseAuthorization();
    app.MapGet("/forgotten", () => "secret");   // no RequireAuthorization, no AllowAnonymous
    await app.StartAsync();

    using var response = await app.GetTestClient().GetAsync("/forgotten");   // no token

    Assert.Equal(expected, response.StatusCode);
}
```

Without a fallback policy the forgotten endpoint answers an anonymous caller with 200. The second case adds one `SetFallbackPolicy` call and the same request gets 401. An endpoint that names its own policy or `[AllowAnonymous]` is not affected, so `/health` in the API stays open on purpose.

3 · Tokens — what JwtBearer checks, and what it renames

`AddJwtBearer` makes the checks a Spring resource server can make — signature, issuer, lifetime and audience — but two defaults differ. With `Authority` set, the handler reads the provider's discovery document, caches its signing keys and, since .NET 8, validates with `JsonWebTokenHandler` (Security token events return a JsonWebToken). The audience check fails closed: an API that sets no audience rejects every token, where Spring checks aud only when you configure it (Spring Security: OAuth 2.0 Resource Server JWT). Lifetime is lenient: `TokenValidationParameters.DefaultClockSkew` is 300 seconds (TokenValidationParameters.DefaultClockSkew) • Verified, five times Spring's 60.

Then the handler renames your claims. `JwtBearerOptions.MapInboundClaims` defaults to `true` (JwtBearerOptions.MapInboundClaims): `sub`, `scp` and `roles` arrive under long WS-Federation-era URIs. Code written against the token as issued — `FindFirst("scp")`, `RoleClaimType = "roles"` — then finds nothing, and a customer with a valid token gets 403 (output 3.6). Set `MapInboundClaims = false` on every scheme (Map, customize and transform claims).

Scopes and roles answer different questions. `scp` lists what a user allowed this client to do, as one space-separated string, and appears only in tokens that carry a user. `roles` is an array holding the app roles assigned to a user *and* the application permissions granted to a daemon (Access token claims reference). A policy must accept each one on purpose, and Microsoft recommends separate role names for users and applications (Verify scopes and app roles).

3.1 · Eight tokens against one set of parameters

C# — samples/L11.TokenLab/Program.cs · #region cases

```
Action<TokenValidationParameters> asConfigured = _ => { };
Action<TokenValidationParameters> skew30s = p => p.ClockSkew = TimeSpan.FromSeconds(30);
Action<TokenValidationParameters> noAudience = p => p.ValidAudience = null;

var cases = new (string Name, string Token, Action<TokenValidationParameters> Adjust)[]
{
    ("valid token",              Mint(),                 asConfigured),
    ("expired 3 min ago",        Mint(expiresMin: -3),   asConfigured),
    ("same token, skew 30 s",    Mint(expiresMin: -3),   skew30s),
    ("audience of another API",  Mint(aud: PartnerApi),  asConfigured),
    ("API sets no audience",     Mint(),                 noAudience),
    ("issuer of another tenant", Mint(iss: OtherTenant), asConfigured),
    ("signed by a stranger key", Mint(key: strangerKey), asConfigured),
    ("unsigned, alg none",       Mint(unsigned: true),   asConfigured),
};

var handler = new JsonWebTokenHandler();
foreach (var (name, token, adjust) in cases)
{
    var p = parameters.Clone();
    adjust(p);
    var result = await handler.ValidateTokenAsync(token, p);
    var verdict = result.IsValid ? "ACCEPTED" : "rejected  " + result.Exception?.GetType().Name;
    Console.WriteLine($"{name,-26}{verdict}");
}
```

Each case changes one thing. `Mint` signs a token that is valid by default; an `Adjust` delegate changes the API's parameters for one case, such as a 30-second skew or no audience at all. The loop reports the exception type the handler returns, which is what an `OnAuthenticationFailed` event would see.

3.2 · What the handler accepted

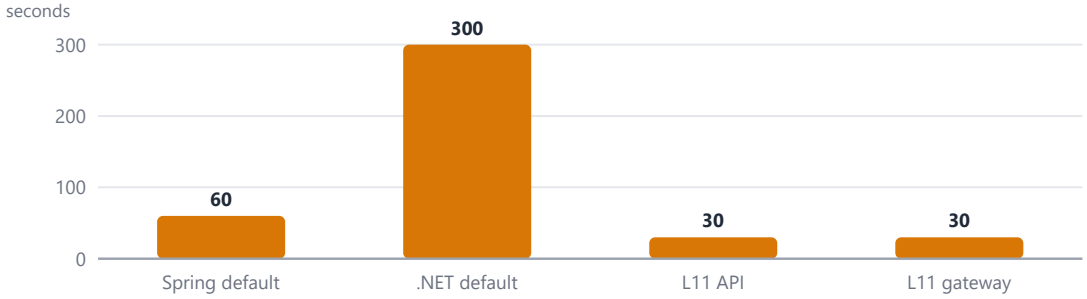
Output — L11.TokenLab — captured on the build machine

```
DefaultClockSkew = 00:05:00

valid token               ACCEPTED
expired 3 min ago         ACCEPTED
same token, skew 30 s     rejected  SecurityTokenExpiredException
audience of another API   rejected  SecurityTokenInvalidAudienceException
API sets no audience      rejected  SecurityTokenInvalidAudienceException
issuer of another tenant  rejected  SecurityTokenInvalidIssuerException
signed by a stranger key  rejected  SecurityTokenInvalidSignatureException
unsigned, alg none        rejected  SecurityTokenInvalidSignatureException
```

Row two is the surprise: a token that expired three minutes ago is accepted. The 30-second skew rejects the same token. Rows five and eight are the reassurance: no configured audience means rejection, and an unsigned `alg: none` token fails signature validation. ● Measured

3.3 · How long after exp a token is still accepted



Spring default verified · .NET default measured by L11.TokenLab · API (both registrations) and gateway parsed from their ClockSkew lines at build

Five minutes is long enough to matter at a gateway. With the default skew, `L11.Gateway` accepted a token that had expired two minutes earlier and proxied it: the gateway test expected 401 and received 502 (no destination exists in the test). Lowering the skew to 30 seconds made it reject the token itself. That comparison was made by hand on 2026-09-16, by commenting the line out — an observation, not a committed test. The bars come from the samples at build ● Measured; Spring's 60 s is ● Verified.

3.4 · Reading scp — the same helper in C# and in Visual Basic

C# samples/L11.SecureQuoteApi/Security/QuoteClaims.cs · #region…

```csharp
public static bool HasScope(this ClaimsPrincipal user,
    string scope)
{
    // Entra puts every delegated scope in ONE claim: "a b c"
    var scp = user.FindFirst(Scope)?.Value;
    if (scp is null)
    {
        return false;
    }
    var scopes = scp.Split(' ',
        StringSplitOptions.RemoveEmptyEntries);
    return scopes.Contains(scope, StringComparer.Ordinal);
}
```

Visual Basic samples/L11.ClaimsVb/Program.vb · #region has-scope

```vb
<Extension>
Function HasScope(user As ClaimsPrincipal,
                  scope As String) As Boolean
    ' Entra puts every delegated scope in ONE claim: "a b c"
    Dim scp = user.FindFirst("scp")?.Value
    If scp Is Nothing Then
        Return False
    End If
    Dim scopes = scp.Split(" "c,
        StringSplitOptions.RemoveEmptyEntries)
    Return scopes.Contains(scope, StringComparer.Ordinal)
End Function
```

Split the one string, then compare ordinally. An exact-value check such as `RequireClaim("scp", "Quotes.Read")` fails for a token whose `scp` is `"Quotes.Read Quotes.Write"`. The VB version is a `<Extension>` function in a module; both read the claim by its raw name, so both depend on mapping being off.

3.5 · Validating in Visual Basic, mapping on and off

Visual Basic samples/L11.ClaimsVb/Program.vb · #region read-user

```vb
Function ReadUser(token As String, key As SecurityKey,
                  mapClaims As Boolean) As ClaimsPrincipal
    Dim handler As New JsonWebTokenHandler With {
        .MapInboundClaims = mapClaims
    }
    Dim parameters As New TokenValidationParameters With {
        .ValidIssuer = Issuer,
        .ValidAudience = Audience,
        .IssuerSigningKey = key,
        .RoleClaimType = "roles"
    }
    ' a console Main cannot Await; blocking here is safe (no
    ' synchronization context) and nowhere else — lesson 05
    Dim result = handler.ValidateTokenAsync(token, parameters).
        GetAwaiter().GetResult()
    Return New ClaimsPrincipal(result.ClaimsIdentity)
End Function
```

A VB library can validate and inspect tokens with the same handler. `MapInboundClaims` is a property of `JsonWebTokenHandler`; `JwtBearerOptions.MapInboundClaims` sets it for you. Blocking on `GetResult()` is acceptable only in a console `Sub Main` — lesson 05 (Async, Tasks & Concurrency).

3.6 · One underwriter token, read twice

Output — L11.ClaimsVb (part 1) captured on the build machine

```
One underwriter access token, read twice

MapInboundClaims = True
  http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier = uw-1
  http://schemas.microsoft.com/identity/claims/scope = Quotes.Read Quotes.Write
  http://schemas.microsoft.com/ws/2008/06/identity/claims/role = Underwriter
  IsInRole("Underwriter") = False
  HasScope("Quotes.Read") = False

MapInboundClaims = False
  sub = uw-1
  scp = Quotes.Read Quotes.Write
  roles = Underwriter
  IsInRole("Underwriter") = True
  HasScope("Quotes.Read") = True
```

With mapping on, the role check and the scope check both return False for a token that carries both. `RoleClaimType = "roles"` no longer matches the renamed claim, and `FindFirst("scp")` finds nothing. Note that `scp` is one claim with two scopes in its value. • **Measured**

3.7 · Delegated tokens and app-only tokens

A user is present — delegated

- Issued by authorization code + PKCE, or by on-behalf-of
- `scp`: one string, the scopes the user consented to for this client
- `sub` is pairwise per application; `oid` is the same across the tenant
- `roles` may also appear: app roles assigned to that user
- In the samples: `owner`, `underwriter`, `owner-read-only`

No user — app-only

- Issued by client credentials, often with a federated assertion
- No `scp`; application permissions arrive in `roles`
- Scope requested is always `…/.default`
- Cannot be exchanged on-behalf-of: there is no user to act for
- In the samples: `back-office-app` — may read any quote, may accept none

4 · Entra ID, External ID and the other authorities

For Entra ID, `Microsoft.Identity.Web` is JwtBearer with Entra's rules applied for you. `AddMicrosoftIdentityWebApi(config.GetSection("AzureAd"))` builds the authority from `Instance` and `TenantId` — a test in `SecurityBehaviourTests` asserts `https://login.microsoftonline.com/{tenant}/v2.0` — and validates the issuer for multi-tenant apps. With the default App ID URI you configure no audience; a custom URI goes in `AzureAd:Audience` (Configure protected web API apps). It also refuses a token that carries neither `scp` nor `roles` unless you opt into ACL-based authorization (Verify scopes and app roles). Version 4.14.2 was the latest on NuGet on 2026-09-16 (microsoft-identity-web) • Verified.

Entra v2 tokens name the API by its client ID. The aud claim of a v2 access token is always the API's client ID, even when the client requested `api://…/Quotes.Read` (Access token claims reference). A hand-written `ValidAudience` copied from the scope string rejects every production token.

Any other OpenID Connect provider goes through plain `AddJwtBearer`, but its claims differ. With RBAC enabled, Auth0 narrows `scope` to the intersection of requested and assigned permissions; it adds a `permissions` array only when its separate "Add Permissions in the Access Token" setting is on too (Auth0 RBAC for APIs). A Cognito user-pool access token carries `scope`, `cognito:groups` and `client_id`, and has an aud claim only when the app requested a resource binding (Cognito access token). Leaving `Audience` unset does not skip that check — JwtBearer then rejects every token, as output 3.2 row 5 shows — so set `ValidateAudience = false` and check `client_id` and `token_use` yourself, in `OnTokenValidated` or a policy requirement. Table 4.1 lines them up.

For customer sign-in on a new system, choose External ID. Azure AD B2C P1 and P2 are no longer sold to new customers, existing tenants keep working, and Microsoft commits to support until at least May 2030. External ID uses the same MSAL code for workforce and customer scenarios (External ID FAQ) • Verified.

4.1 · The same API behind five authorities

Authority	Scopes arrive in	Roles or groups arrive in	.NET registration	Watch for
Entra ID (workforce, v2 tokens)	`scp` — one string	`roles` — array; `groups` (object IDs) when configured	`AddMicrosoftIdentityWebApi`	aud = client ID; more than 200 groups become an overage claim
Entra External ID	`scp`	`roles`	`AddMicrosoftIdentityWebApi`, external tenant	Register the API in an *external* tenant; new customer systems start here, not on B2C
Azure AD B2C	`scp`	Claims your user flow or custom policy adds	`AddMicrosoftIdentityWebApi`	• Closed to new customers supported to at least May 2030
Auth0	`scope`	`permissions` array when RBAC and "Add Permissions in the Access Token" are on	`AddJwtBearer`	`scope` is the intersection of requested and assigned
Amazon Cognito	`scope`	`cognito:groups`	`AddJwtBearer`	No aud without a resource binding — set `ValidateAudience = false`, then check `client_id` and `token_use`

4.2 · Any OpenID Connect authority

C# samples/L11.SecureQuoteApi/Security/AuthenticationSetup.cs · #region jwt-bearer

```
// Any OpenID Connect authority: Entra ID, External ID, Auth0, Cognito, Keycloak
public static IServiceCollection AddQuoteApiJwt(this IServiceCollection services,
    IConfiguration config)
{
    var identity = config.GetSection("Identity");
    services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(o =>
        {
            o.Authority = identity["Authority"];   // discovery document + JWKS signing keys
            o.Audience = identity["Audience"];     // the aud claim must name THIS API
            o.MapInboundClaims = false;            // keep sub, scp, roles as issued
            o.TokenValidationParameters.NameClaimType = "name";
            o.TokenValidationParameters.RoleClaimType = "roles";
            o.TokenValidationParameters.ClockSkew = TimeSpan.FromSeconds(30);   // default 5 min
        });
    return services;
}
```

Four settings carry the lesson so far. `Authority` for discovery and keys, `Audience` for this API, `MapInboundClaims = false` for raw claim names and a 30-second `ClockSkew`. For Cognito you would set `ValidateAudience = false` and check `client_id` and `token_use` in `OnTokenValidated`: leaving `Audience` unset alone rejects every token (3.2). The rest is unchanged.

4.3 · Entra ID through Microsoft.Identity.Web

C# samples/L11.SecureQuoteApi/Security/AuthenticationSetup.cs · #region entra

```
// Entra ID through Microsoft.Identity.Web: AzureAd:Instance + TenantId + ClientId
public static IServiceCollection AddQuoteApiEntra(this IServiceCollection services,
    IConfiguration config)
{
    services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddMicrosoftIdentityWebApi(config.GetSection("AzureAd"))
        .EnableTokenAcquisitionToCallDownstreamApi()  // OBO + client credentials
        .AddDownstreamApi("PartnerRates", config.GetSection("DownstreamApis:PartnerRates"))
        .AddInMemoryTokenCaches();              // several instances: AddDistributedTokenCaches

    // the library keeps its own defaults: repeat the claim and skew settings of 4.2
    services.Configure<JwtBearerOptions>(JwtBearerDefaults.AuthenticationScheme, o =>
    {
        o.MapInboundClaims = false;                              // same claim names as above
        o.TokenValidationParameters.NameClaimType = "name";
        o.TokenValidationParameters.RoleClaimType = "roles";
        o.TokenValidationParameters.ClockSkew = TimeSpan.FromSeconds(30);   // still 5 min
    });
    services.AddScoped<PartnerRatesClient>();
    return services;
}
```

One chain registers validation and token acquisition, and the library keeps its own defaults for claims and skew. The `Configure<JwtBearerOptions>` call runs after the library's own configuration and repeats the claim and clock-skew settings of 4.2: without it the skew would still be five minutes. With it the policies work unchanged under either provider, and a test asserts the authority, the claim mapping and the skew. The in-memory token cache holds downstream tokens per instance; an API that runs as several ECS tasks needs `AddDistributedTokenCaches` (Call downstream APIs from web APIs).

5 · Flows — who gets a token for whom

Choose the flow by asking who is present: a user at a client, a user behind another API, or no user at all. A user at a browser, phone or desktop gets a delegated token through authorization code with PKCE. The implicit grant and the password grant are both flows Microsoft recommends against (Implicit grant flow · ROPC grant).

An API that calls another API as the same user exchanges the token it received. That is the on-behalf-of flow: the quote API sends the customer's token to Entra ID and gets a token for the partner-rates API that still names the customer. Only delegated scopes travel; the flow works for user tokens only, and a single-page app hands its token to a confidential middle tier rather than doing the exchange itself (On-behalf-of flow).

A job with no user uses client credentials, and should not need a secret. The token carries application permissions in `roles`. Microsoft.Identity.Web reads its credential from configuration and ranks client secrets as suitable for development and testing only (Credentials overview for Microsoft.Identity.Web). Workload identity federation removes the stored credential:

Entra trusts a token from the platform the job runs on — GitHub Actions, Kubernetes including EKS, or AWS through IAM outbound identity federation (Workload identity federation · IAM outbound identity federation). This lesson's `appsettings.json` holds only a file path to such a token.

5.1 · Six flows and credentials for the MotorQuote platform

identity view

1 Authorization code + PKCE

architecture · Same idea

A user signs in through a browser, mobile or desktop client; the client gets a delegated token.

TOKEN	`scp` scopes, a user `sub` and `oid`
.NET	MSAL in the client; `AddOpenIdConnect` in a server-rendered app, which uses pushed authorization requests from .NET 9 when the provider advertises them (Configure OpenID Connect web authentication)
RETIRE	Implicit grant — Microsoft recommends against it and silent renewal breaks without third-party cookies (Implicit grant flow)
RETIRE	Password grant (ROPC) — incompatible with MFA (ROPC grant)

2 On-behalf-of

architecture · Different

An API calls another API as the same user by exchanging the token it received.

NEEDS	A confidential client; the incoming token's `aud` must be this API
CARRIES	Delegated scopes only — app roles stay with the user, never the middle tier
LIMIT	User tokens only: an app-only token cannot be exchanged (On-behalf-of flow)
.NET	`EnableTokenAcquisitionToCallDownstreamApi` + `IDownstreamApi.GetForUserAsync` (5.3)

5.1 · Six flows and credentials (continued)

identity view

3 Client credentials

architecture · Same idea

A daemon or batch job calls an API as itself; there is no user in the token.

TOKEN	Application permissions in `roles`; no `scp`
SCOPE	Always `api://…/.default` — the permissions granted to the app
.NET	`GetForAppAsync`, or MSAL `AcquireTokenForClient`
WATCH	Users and apps share the `roles` claim: give them different role names (Verify scopes and app roles)
ASSIGN	Unless the API requires assignment, any app in the tenant can get a token with no `roles` — never accept "authenticated" alone (Client credentials flow)

4 Workload identity federation

cloud · Different

A workload outside Azure proves who it is with a token from its own platform instead of a secret.

FROM	GitHub Actions, any Kubernetes cluster including EKS, Google Cloud, SPIFFE, and AWS workloads through IAM outbound identity federation (Workload identity federation)
EXCHANGE	The platform's JWT becomes the client assertion of a client-credentials request
.NET	`ClientCredentials` with `SourceType: SignedAssertionFilePath` in `appsettings.json` (Credentials overview for Microsoft.Identity.Web)
MATCH	`issuer`, `subject` and `audience` are compared case-sensitively

5.1 · Six flows and credentials (continued)

identity view

5 Gateway token relay

architecture · Renamed

A YARP gateway validates the bearer token, applies a route policy and forwards the request.

FORWARDS	The same `Authorization` header — the API validates it again
CHECKS	Coarse route policies only: it cannot see the quote a request touches
NOT	Windows / Negotiate identity: it is bound to a connection and does not flow through the proxy (YARP authentication and authorization)
LIKE	Spring Cloud Gateway or Zuul with a token relay filter

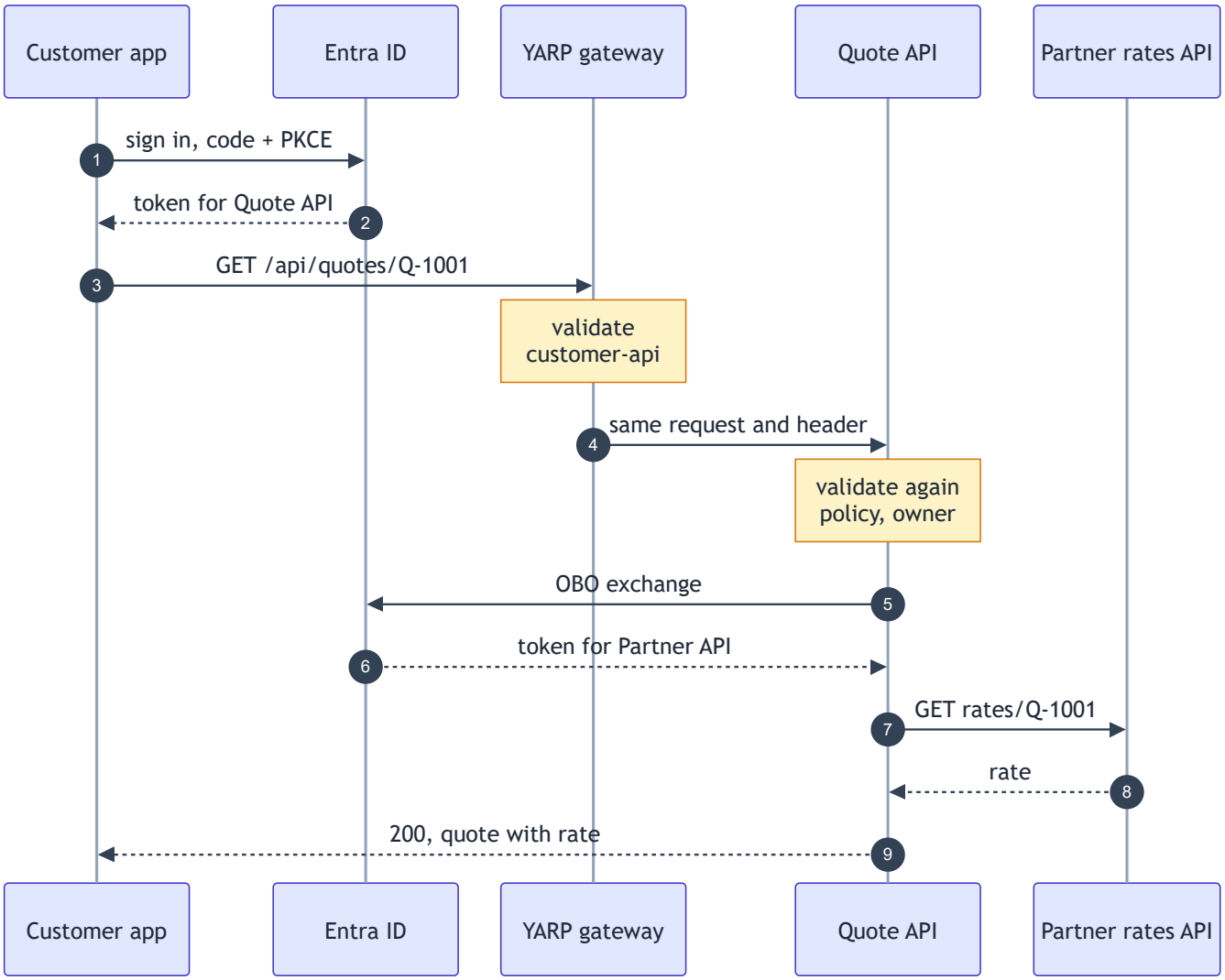
6 Secrets you should not have

cloud · Trap

Client secrets are the weakest credential; the guidance ranks them last.

SECRET	Development and testing only (Credentials overview for Microsoft.Identity.Web)
CERT	Production outside Azure when federation is not available
FED.	No stored credential to leak, rotate or let expire
AWS	IAM outbound identity federation issues the JWT: default 300 s, 60–3,600 s allowed (IAM outbound identity federation)

5.2 · A customer quote request that calls a rating partner



indigo = participants · amber notes = where a token is validated · steps 1–2 happen once per sign-in; steps 5–6 only when no cached partner token is still valid

5.3 · Calling the partner as the user, and as the API

C# — samples/L11.SecureQuoteApi/Partners/PartnerRatesClient.cs · #region downstream

```
// On behalf of the caller: the incoming token is exchanged (OBO) for a
// partner token that still names the customer. Scopes: DownstreamApis:PartnerRates
public Task<PartnerRate?> RateForCustomerAsync(string quoteId, CancellationToken ct) =>
    api.GetForUserAsync<PartnerRate>("PartnerRates",
        o => o.RelativePath = $"rates/{quoteId}",
        cancellationToken: ct);

// As the API itself (client credentials): no user in the token, app roles only
public Task<PartnerRate?> RateForBatchAsync(string quoteId, CancellationToken ct) =>
    api.GetForAppAsync<PartnerRate>("PartnerRates", o =>
    {
        o.RelativePath = $"batch-rates/{quoteId}";
        o.Scopes = ["api://l11-partner-api/.default"];   // app tokens: .default only
    }, ct);
```

The two calls differ in one word, and in who the partner sees. `GetForUserAsync` runs on-behalf-of with the scopes from `DownstreamApis:PartnerRates`; `GetForAppAsync` runs client credentials with `.default`. Both compile and are registered when `Identity:Provider` is `Entra`; neither runs in the tests, because acquiring a token needs a real tenant.

5.4 · A federated credential is a file path

JSON — L11.SecureQuoteApi/appsettings.json · "AzureAd"

```
"AzureAd": {
  "Instance": "https://login.microsoftonline.com/",
  "TenantId": "00000000-0000-0000-0000-00000000a011",
  "ClientId": "00000000-0000-0000-0000-00000000c011",
  "ClientCredentials": [
    {
      "SourceType": "SignedAssertionFilePath",
      "SignedAssertionFileDiskPath": "/var/run/secrets/tokens/entra-federation"
    }
  ]
}
```

No secret is stored: `SignedAssertionFilePath` names a file that holds a token the platform issued. The pod or job finds that token at the path — for example a projected service-account token in Kubernetes — and Microsoft.Identity.Web sends it as the client assertion, so there is nothing to rotate. Tenant and client IDs are zero-filled placeholders and the path is illustrative (Credentials overview for Microsoft.Identity.Web).

6 · Authorization design — policies, resources and the gateway

Put each decision where the data it needs is available. A gateway sees only the token and the path, so it can check “a valid token” and “an underwriter”. An endpoint policy sees the token and the operation, so it checks scopes and roles. Only the endpoint that has loaded the quote can check “is this caller its owner”.

Resource-based authorization is how .NET asks about one object. The endpoint loads the quote and calls `IAuthorizationService.AuthorizeAsync(user, quote, QuoteOperations.Read)`; an `AuthorizationHandler<OperationAuthorizationRequirement, Quote>` decides (Resource-based authorization). The owner comes from the token's `sub`, never from the request body — the Spring equivalent is a `PermissionEvaluator` behind `hasPermission`.

A YARP gateway reuses the same policies, per route, from configuration. Each route names an `AuthorizationPolicy`; `default` and `anonymous` are special values, and a route with no policy is not authorized at all unless a fallback policy exists. Bearer headers flow to the destination unchanged, so the API validates the token again (YARP authentication and authorization). Treat the gateway as the first filter, not the only one: in the status matrix the API alone decides every 403 that depends on who owns a quote.

6.1 · Access rules — Spring Security paths vs ASP.NET Core named policies

Kotlin — for comparison the paths-first shape you know

```kotlin
// Spring Security 6 Kotlin DSL. A JWT converter
// maps scp to SCOPE_ and roles to ROLE_ authorities.
@Bean
fun api(http: HttpSecurity): SecurityFilterChain {
    http {
        authorizeHttpRequests {
            authorize("/health", permitAll)
            authorize(HttpMethod.GET, "/quotes/**",
                hasAnyAuthority("SCOPE_Quotes.Read",
                    "ROLE_Quotes.Read.All"))
            authorize(HttpMethod.POST, "/quotes/**",
                hasAuthority("SCOPE_Quotes.Write"))
            authorize("/underwriting/**",
                hasRole("Underwriter"))
            authorize(anyRequest, authenticated)
        }
        oauth2ResourceServer { jwt { } }
    }
    return http.build()
}
```

C# samples/L11.SecureQuoteApi/Program.cs · #region policies

```csharp
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("quotes.read", p => p.RequireAssertion(c =>
        c.User.HasScope(ReadScope)
        || c.User.IsInRole(ReadAllPermission)))
    .AddPolicy("quotes.write", p => p.RequireAssertion(c =>
        c.User.HasScope(WriteScope)))
    .AddPolicy("underwriting", p => p
        .RequireRole(UnderwriterRole)
        .RequireAssertion(c => c.User.HasScope(ReadScope)))
    // endpoints without RequireAuthorization are NOT public
    .SetFallbackPolicy(new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build());

builder.Services.AddSingleton<IAuthorizationHandler,
    QuoteAuthorizationHandler>();
```

Spring maps paths to rules in one place; ASP.NET Core names the rules in one place and attaches them to endpoints. `SetFallbackPolicy` is `anyRequest().authenticated()`. The underwriting policy demands the role *and* a delegated scope, so an app-only token holding an `Underwriter` role would still be refused. The handler registered last answers the per-quote question in 6.2 and 6.3.

6.2 · Policy first, then the resource check

C# samples/L11.SecureQuoteApi/Program.cs · #region read-endpoint

```csharp
app.MapGet("/health", () => "ok").AllowAnonymous();

var quotes = app.MapGroup("/quotes");

quotes.MapGet("/{id}", async (string id, ClaimsPrincipal user,
    QuoteStore store, IAuthorizationService authz) =>
{
    if (store.Find(id) is not { } quote)
    {
        return Results.NotFound();
    }
    // the policy said "may read quotes"; this asks "may read THIS quote"
    var result = await authz.AuthorizeAsync(user, quote, QuoteOperations.Read);
    return result.Succeeded ? Results.Ok(quote) : Results.Forbid();
}).RequireAuthorization("quotes.read");
```

Two checks, two kinds of 403. `RequireAuthorization("quotes.read")` rejects a token without the scope before the handler body runs; `AuthorizeAsync`, answered by the handler in 6.3, rejects the right scope on someone else's quote. Here existence is checked first, so a caller with the read scope can tell a missing quote (404) from another customer's (403) — harmless for opaque IDs, an enumeration leak for sequential ones like `Q-1001`.

6.3 · Who may read or accept this quote

C# samples/L11.SecureQuoteApi/Security/QuoteAuthorizationHandler.cs · #region owner-handler

```csharp
// "May this caller do this operation to THIS quote?" — asked after the policy passed
public sealed class QuoteAuthorizationHandler
    : AuthorizationHandler<OperationAuthorizationRequirement, Quote>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        OperationAuthorizationRequirement requirement,
        Quote quote)
    {
        var user = context.User;
        var isOwner = user.SubjectId() == quote.OwnerId;

        var allowed = requirement.Name switch
        {
            // read: the owner, any underwriter, or a back-office app
            nameof(QuoteOperations.Read) => isOwner
                || user.IsInRole(QuoteClaims.UnderwriterRole)
                || user.IsInRole(QuoteClaims.ReadAllPermission),
```

```
            // accept: the owner only — nobody accepts a quote for a customer
            nameof(QuoteOperations.Accept) => isOwner,
            _ => false,
        };

        if (allowed)
        {
            context.Succeed(requirement);
        }
        // no context.Fail(): another handler may still succeed the requirement
        return Task.CompletedTask;
    }
}
```

A handler succeeds or stays silent. Leaving the requirement unmet is enough to deny; calling `context.Fail()` would also stop any other handler from granting it. Accepting a quote is an owner-only operation, so this handler refuses `other-customer` and `underwriter-rw` in matrix 9.3: both hold the write scope. The plain `underwriter` and the `back-office-app` never reach it — with no `Quotes.Write` scope, policy `quotes.write` stops them first.

6.4 · The YARP gateway: coarse checks per route

C# — samples/L11.Gateway/Program.cs · #region gateway-policies

```
// coarse checks only: "a valid token" and "an underwriter" — the API still decides per quote
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("customer-api", p => p.RequireAuthenticatedUser())
    .AddPolicy("underwriting", p => p.RequireRole("Underwriter"));

builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.MapReverseProxy();   // every route names its AuthorizationPolicy in appsettings.json
app.Run();
```

The gateway defines two policies and lets every route pick one by name. Above this panel in `Program.cs` sits a second JwtBearer registration that repeats the API's claim settings — mapping off, `roles` as the role claim, the same clock skew — or its role check would read a different claim than the API does. The gateway can ask only “valid token?” and “underwriter?”: the quote a request touches is invisible to it.

6.5 · Routes — Spring Cloud Gateway vs YARP

Spring Cloud Gateway — for comparison — the routes you know

```
# Spring Cloud Gateway application.yml, routes only
routes:
  - id: quotes
    uri: http://quote-api:8080
    predicates: [ "Path=/api/quotes/**" ]
    filters: [ "StripPrefix=1" ]
  - id: underwriting
    uri: http://quote-api:8080
    predicates: [ "Path=/api/underwriting/**" ]
    filters: [ "StripPrefix=1" ]
  - id: health
    uri: http://quote-api:8080
    predicates: [ "Path=/health" ]
# who may call each route is not here: a security
# filter chain matches the paths a second time (6.1)
```

JSON — L11.Gateway/appsettings.json · "Routes"

```
"Routes": {
  "quotes": {
    "ClusterId": "quote-api",
    "AuthorizationPolicy": "customer-api",
    "Match": { "Path": "/api/quotes/{**rest}" },
    "Transforms": [ { "PathRemovePrefix": "/api" } ]
  },
  "underwriting": {
    "ClusterId": "quote-api",
    "AuthorizationPolicy": "underwriting",
    "Match": { "Path": "/api/underwriting/{**rest}" },
    "Transforms": [ { "PathRemovePrefix": "/api" } ]
  },
  "health": {
    "ClusterId": "quote-api",
    "AuthorizationPolicy": "anonymous",
    "Match": { "Path": "/health" }
  }
}
```

YARP puts the authorization rule on the route. Each route names an `AuthorizationPolicy` — `customer-api`, `underwriting` or the built-in `anonymous` for `/health` — and YARP reloads the configuration without a restart (YARP authentication and authorization). Spring Cloud Gateway routes only match and rewrite; the rule lives in a separate filter chain. The `ClusterId` points at a cluster of destinations defined below these routes in the same file. YARP returns in lesson 12 (Cloud-Native .NET on AWS & Modernization) for incremental migration.

7 · Secrets and hardening — STRIDE for one API

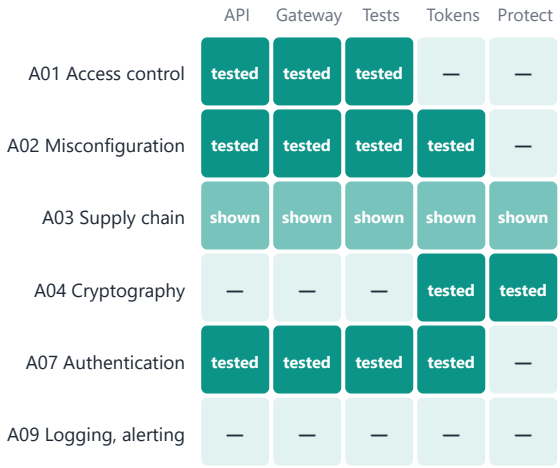
Secrets leave the repository, and where possible they stop existing. `dotnet user-secrets` keeps development values in an unencrypted JSON file in your profile and is added to configuration only in the Development environment (Safe storage of app secrets in development). In production, AWS Systems Manager Parameter Store — including Secrets Manager secrets under `/aws/reference/secretsmanager/` — plugs in as a configuration provider (aws-dotnet-extensions-configuration); on Azure the Key Vault configuration provider does the same and authenticates with a managed identity (Azure Key Vault configuration provider). Better still is no secret at all: a federated credential (section 5).

Data Protection is the `<machineKey>` you now own. It encrypts authentication cookies, antiforgery tokens and any payload you protect yourself, with AES-256-CBC and HMACSHA256 keys that rotate every 90 days (Data Protection key management and lifetime) • Verified. Keys are isolated by purpose string and application name, and a process with no shared key store has a ring of its own — so a cookie issued by one ECS task is unreadable on the next. Persist the ring to one store; on AWS, `PersistKeysToAWSSystemsManager` does it (aws-ssm-data-protection-provider-for-aspnet). `L11.ProtectLab` measures all four failure modes (7.4).

Transport and browser controls are narrower for an API than for a site. An API should not listen on HTTP or should reject it; a redirect arrives after the client has sent its token in clear, and HSTS is an instruction only browsers obey (Enforce HTTPS). CORS is an allowlist for browsers, antiforgery matters only for cookie-authenticated form posts, and rate limiting partitions by caller.

Supply chain and static analysis are build gates, not reports. For `net10.0` projects NuGet audits transitive packages by default (Auditing package dependencies); this repository keeps advisories as warnings so an old lesson does not break overnight — lesson 07 (Solutions, MSBuild & Mono-repos). CodeQL analyses C# up to version 14 and ASP.NET Core but lists no Visual Basic support (CodeQL supported languages), so a VB estate needs another analyser: SonarQube Server and Community Build both analyse VB.NET (SonarQube: VB.NET analysis). Chart 7.1 scores these samples against the OWASP Top 10, and table 7.2 applies STRIDE to the quote API.

7.1 · OWASP Top 10:2025 — what the lesson 11 samples cover



tested = asserted by a test or printed by a lab · shown = configured, not asserted · — = not covered · a rubric, not a benchmark · A05, A06, A08 and A10 not scored

The empty row is the honest one. The samples log nothing when a request is denied, and no test would notice. The hooks are small and official: `JwtBearerEvents.OnAuthenticationFailed` fires when a token fails validation and `OnForbidden` when authorization ends in a 403 (JwtBearer Events); an `IAuthorizationMiddlewareResultHandler` also sees the failed requirements of a policy (Authorization middleware result handler). Log the subject, path and reason with a structured message template — never the token. Where structured logs go is the CloudWatch example of lesson 12 (Cloud-Native .NET on AWS & Modernization); an alarm on the rate of 401 and 403 answers is left to your monitoring, and this track builds none. Supply chain is "shown" only: NuGet audit runs on every restore, no test asserts it. Categories from OWASP Top 10:2025; the scores are the author's judgement • Estimate

7.2 · STRIDE for the quote API — threat, control, evidence

Threat	Against MotorQuote	Control	Evidence in the samples
Spoofing	A token from another tenant, another API or a forged key	Issuer, audience and signature validation; no `alg: none`	Output 3.2 rows 4 and 6–8 · matrix row `wrong-audience` = 401
Tampering	An edited token or share link	JWT signature; Data Protection MAC	Output 3.2 stranger key · output 7.5 one character changed

Threat	Against MotorQuote	Control	Evidence in the samples
Repudiation	"I never accepted that quote"	The owner comes from the validated `sub`, never the body; log `sub` with the decision	Owner check in 6.3 · decision logging **• not in the samples**
Information disclosure	Customer B reads customer A's quote	Resource-based handler after the scope policy	Matrix row `other-customer` = 403 on read
Denial of service	One caller floods `GET /quotes/{id}`	Fixed-window limiter partitioned by `sub`	Throttling test: 200, 200, 429
Elevation of privilege	A customer lists referrals; an app accepts a quote	Role + scope policy; accept is owner-only	Matrix rows `owner` = 403 on referrals, `back-office-app` = 403 on accept

7.3 · CORS and a per-caller rate limit

C# — samples/L11.SecureQuoteApi/Program.cs · #region hardening

```
builder.Services.AddCors(o => o.AddDefaultPolicy(p => p
    .WithOrigins(config.GetSection("Cors:Origins").Get<string[]>() ?? [])
    .WithMethods("GET", "POST")
    .WithHeaders("Authorization", "Content-Type")));

// behind a load balancer the socket peer is the balancer, not the caller:
// trust X-Forwarded-For only from the balancer's own network
builder.Services.Configure<ForwardedHeadersOptions>(o =>
{
    o.ForwardedHeaders = ForwardedHeaders.XForwardedFor;
    if (config["Proxy:Network"] is { } cidr)
    {
        o.KnownIPNetworks.Add(System.Net.IPNetwork.Parse(cidr));
    }
});

builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    // one bucket per token subject; callers without one share their IP's bucket
    o.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(http =>
        RateLimitPartition.GetFixedWindowLimiter(
            http.User.SubjectId() ?? http.Connection.RemoteIpAddress?.ToString() ?? "anonymous",
            _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = config.GetValue("RateLimit:PerMinute", 60),
                Window = TimeSpan.FromMinutes(1),
            }));
});
```

Both are allowlists. CORS names origins, methods and headers and never calls `AllowCredentials`, because this API reads bearer tokens, not cookies. The limiter partitions by the token's subject and falls back to the client IP. **Behind a load balancer that IP is the balancer's,** so every anonymous caller shares one bucket. `UseForwardedHeaders` swaps in the `X-Forwarded-For` value, but only for a request that arrives from a known proxy or network, and by default only loopback is known (Configure ASP.NET Core to work with proxy servers and load balancers). Add the balancer's subnet — `Proxy:Network` here — and run the middleware before the others. `ForwardedHeadersTests` asserts both outcomes: with the network trusted the second caller gets 200, without it 429.

7.4 · Five ways to read one protected share link

C# — samples/L11.ProtectLab/Program.cs · #region protect

```
var instanceA = NewKeyRing();
var shareLinks = instanceA.CreateProtector("quotes.share-link")
    .ToTimeLimitedDataProtector();

var link = shareLinks.Protect("Q-1001", TimeSpan.FromMinutes(10));
var expired = shareLinks.Protect("Q-1001", DateTimeOffset.UtcNow.AddSeconds(-1));
var tampered = link[..^4] + (link[^4] == 'A' ? 'B' : 'A') + link[^3..];

var attempts = new (string Name, Func<string> Read)[]
{
    ("same purpose, same key ring", () => shareLinks.Unprotect(link)),
    ("other purpose", () => instanceA.CreateProtector("quotes.accept-link")
        .ToTimeLimitedDataProtector().Unprotect(link)),
    ("one character changed", () => shareLinks.Unprotect(tampered)),
```

```
    ("expired a second ago", () => shareLinks.Unprotect(expired)),
    ("another instance's key ring", () => NewKeyRing().CreateProtector("quotes.share-link")
        .ToTimeLimitedDataProtector().Unprotect(link)),
};
```

One success, four failures, and three of the failures are deliberately indistinguishable. `ToTimeLimitedDataProtector` adds an expiry; a purpose string scopes a payload to one use. A wrong purpose, a changed character and a foreign key ring all raise the same `CryptographicException`; only expiry says why. `NewKeyRing()` is in-memory so the lab writes nothing to disk — production persists and shares keys (Configure Data Protection).

7.5 · What Data Protection returned

Output — L11.ProtectLab captured on the build machine

```
share link for Q-1001: 104 chars, starts CfDJ8

same purpose, same key ring  Q-1001
other purpose                rejected: Error occurred during a cryptographic operation
one character changed        rejected: Error occurred during a cryptographic operation
expired a second ago         rejected: The payload expired
another instance's key ring  rejected: Error occurred during a cryptographic operation
```

The last row is the production incident. Two instances with separate key rings cannot read each other's payloads — behind a load balancer, users are signed out at random. `CfDJ8` is the payload's magic header, the same on every run. • Measured

8 · VB and legacy — Forms and Windows authentication

Legacy .NET security is claims-based underneath, which makes the code migration smaller than the configuration migration. Since .NET Framework 4.5, `GenericPrincipal` — what Forms authentication code typically put on the thread — inherits from `ClaimsPrincipal` (GenericPrincipal). Its roles surface as the same `…/claims/role` claim type the default JwtBearer mapping produces (output 8.4), so a VB rule written against `IsInRole` keeps working when handed a modern principal.

The plumbing mostly does not move. `<authentication mode="Forms">`, Membership tables, `<machineKey>` and static `ClaimsPrincipal.Current` have no .NET 10 equivalent; table 8.1 maps each to its replacement. Windows authentication is the exception: an app hosted on IIS still switches it on in `web.config`, Kestrel and HTTP.sys use the Negotiate handler, and Microsoft positions it for intranets where users and servers share a domain (Configure Windows Authentication).

.NET 10 changed one cookie default that mixed MVC-and-API estates will notice. Cookie authentication now answers unauthenticated calls to known API endpoints with 401 and 403 instead of redirecting to a login page (Cookie login redirects disabled for API endpoints) — right for a JavaScript client, surprising for a page that relied on the redirect.

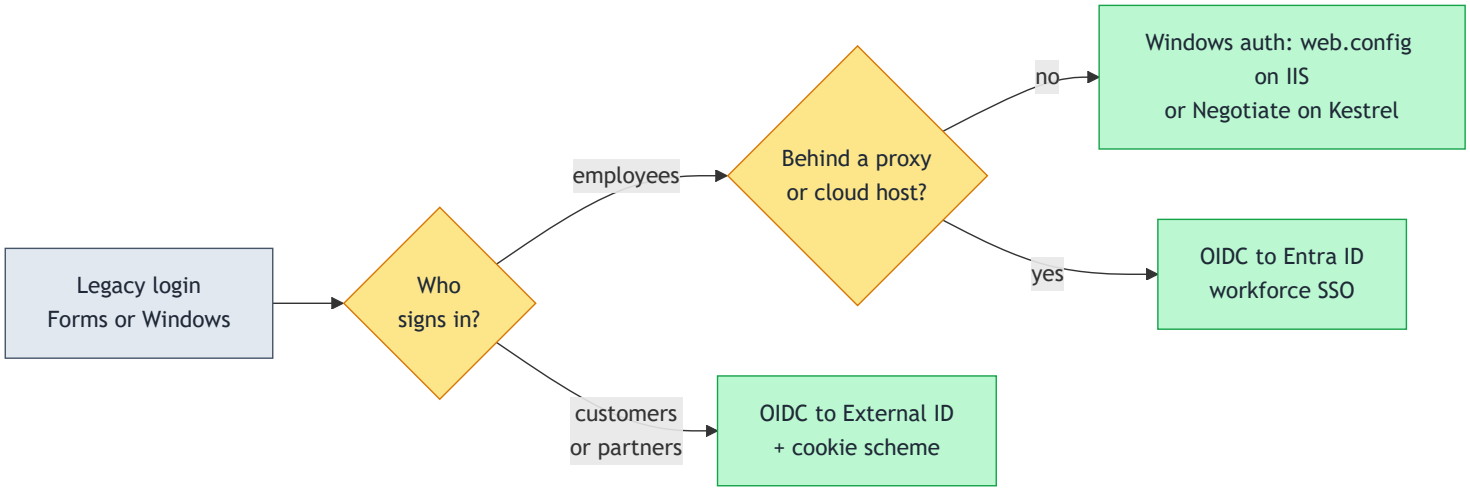
Old and new applications can share one sign-in during a strangler migration. The System.Web adapters' remote authentication, or OWIN cookie middleware reading a Data Protection ring shared with the new app, keeps users signed in across both (Share authentication cookies among ASP.NET apps). That migration belongs to lesson 12 (Cloud-Native .NET on AWS & Modernization), and moving the VB code itself to lesson 06 (VB.NET for Legacy Estates). Visual Basic has no ASP.NET Core templates, so VB stays in libraries that receive a `ClaimsPrincipal` from a C# host.

8.1 · .NET Framework security pieces and their .NET 10 replacements

.NET Framework on IIS	ASP.NET Core on .NET 10	Kind	Migration note
Forms authentication, `FormsAuthentication.SetAuthCookie`	Cookie scheme + OpenID Connect to Entra ID or External ID	• Different	Stop owning passwords; the IdP signs users in
IIS Windows authentication, `WindowsPrincipal`	`web.config` on IIS as before; `AddNegotiate()` on Kestrel or HTTP.sys; or Entra SSO	• Same idea	A Windows identity does not flow through YARP to the API
ASP.NET Membership / SimpleMembership tables	ASP.NET Core Identity, or the external IdP	• Different	Different schema: migrate users, or move them to the IdP
`<machineKey>`	Data Protection key ring	• Renamed	Persist and share the ring; keys rotate every 90 days
`ClaimsPrincipal.Current`, `Thread.CurrentPrincipal`	`HttpContext.User`, or a `ClaimsPrincipal` parameter	• Trap	Not set by default: static access finds no user
`User.IsInRole` in code-behind	Policies; `IsInRole` still works	• Same idea	Check which claim type the role arrives under

.NET Framework on IIS	ASP.NET Core on .NET 10	Kind	Migration note
Shared login across old and new apps	System.Web adapters remote auth, or shared cookies	• Different	Temporary by design — lesson 12 (Cloud-Native .NET on AWS & Modernization)

8.2 · Choosing the replacement for a legacy login



amber = question · slate = legacy starting point · green = target · while old IIS pages still serve users, bridge the sign-in with System.Web adapters or a shared cookie (table 8.1)

8.3 · A Forms-era principal read as claims, in Visual Basic

Visual Basic — `samples/L11.ClaimsVb/Program.vb · #region legacy-principal`

```
' What Web Forms code found in Thread.CurrentPrincipal after a
' Forms login: a GenericPrincipal. Since .NET 4.5 it IS a
' ClaimsPrincipal, so no conversion is needed.
Dim legacy As New GenericPrincipal(
    New GenericIdentity("uw-1", "Forms"), {"Underwriter"})
Dim asClaims As ClaimsPrincipal = legacy
For Each c In asClaims.Claims
    Console.WriteLine($"  {c.Type} = {c.Value}")
Next
Console.WriteLine(
    $"  IsInRole(""Underwriter"") = {asClaims.IsInRole("Underwriter")}")
```

> **No conversion, no adapter.** The widening assignment to `ClaimsPrincipal` compiles under `Option Strict On` because a `GenericPrincipal` already is one. The loop is the same code a .NET 10 library would run over `HttpContext.User`.

8.4 · The legacy role, under the long claim type

Output — L11.ClaimsVb (part 2) — captured on the build machine

```
A Forms-era principal, read as claims
  http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name = uw-1
  http://schemas.microsoft.com/ws/2008/06/identity/claims/role = Underwriter
  IsInRole("Underwriter") = True
```

> **The role claim type printed here is exactly what `MapInboundClaims = true` produced in 3.6.** That mapping exists so tokens look like the principals .NET Framework code already understood — and new code turns it off. • Measured

9 · Hands-on — test-signed tokens and a 401/403/200 matrix

Test security with real tokens, not with a bypass. Replacing authentication with a fake handler that always succeeds — the .NET cousin of `@WithMockUser` — exercises policies but never token validation, which is where sections 2 and 3 found the bugs. These tests sign JWTs with an RSA key made for the test run and point JwtBearer at that key, so issuer, audience, lifetime and signature are validated exactly as in production (Integration tests in ASP.NET Core).

The matrix is a threat model you can run. 9 callers — anonymous, expired, wrong audience, the owner, another customer, a read-only owner, an underwriter, an underwriter who also holds the write scope and a back-office application — against 4 endpoints gives 36 cases with an expected status each. A change that opens a hole flips a cell and fails the build; the same file is the reviewable specification of who may do what.

Run everything, then compare with what you saw here. 49 tests across 5 classes, three console labs and two web projects build on SDK 10.0.401 with warnings as errors.

9.1 · Minting test tokens

C# — samples/L11.SecureQuoteApi.Tests/TestTokens.cs · #region test-tokens

```
public static class TestTokens
{
    public const string Issuer = "https://login.example.test/tenant-l11/v2.0";
    public const string Audience = "api://l11-secure-quote-api";

    // One RSA key per test run: the API under test trusts it, nothing else ever sees it
    public static readonly RsaSecurityKey Key = new(RSA.Create(2048)) { KeyId = "l11-test" };

    public static string Create(string? sub = null, string? scp = null, string[]? roles = null,
        string audience = Audience, TimeSpan? expiresIn = null)
    {
        var claims = new Dictionary<string, object>();
        if (sub is not null) { claims["sub"] = sub; }
        if (scp is not null) { claims["scp"] = scp; }         // "Quotes.Read Quotes.Write"
        if (roles is not null) { claims["roles"] = roles; }   // a JSON array, as Entra issues

        var issued = DateTime.UtcNow.AddMinutes(-10);
        return new JsonWebTokenHandler().CreateToken(new SecurityTokenDescriptor
        {
            Issuer = Issuer,
            Audience = audience,
            Claims = claims,
            IssuedAt = issued,
            NotBefore = issued,
            Expires = DateTime.UtcNow + (expiresIn ?? TimeSpan.FromMinutes(5)),
            SigningCredentials = new SigningCredentials(Key, SecurityAlgorithms.RsaSha256),
        });
    }
}
```

The claim shapes copy Entra's. `scp` is one string and `roles` a JSON array, so a test fails for the same reasons production would. Issuer and audience are those of the generic OpenID Connect setup in 4.2, where an `api://` audience is legitimate; an Entra v2 token would carry the client ID (4). The RSA key exists only in the test process; issuing tokens ten minutes in the past lets a test set an expiry that is already over.

9.2 · Pointing both apps at the test key

C# — samples/L11.SecureQuoteApi.Tests/TestKeyFactory.cs · #region factory

```
public class TestKeyFactory<TEntryPoint> : WebApplicationFactory<TEntryPoint>
    where TEntryPoint : class
{
    public bool MapInboundClaims { get; init; }   // false = what both apps configure

    protected override void ConfigureWebHost(IWebHostBuilder builder) =>
        builder.ConfigureTestServices(services =>
            services.Configure<JwtBearerOptions>(JwtBearerDefaults.AuthenticationScheme, o =>
            {
                // No discovery call: trust the local test key instead of the authority's JWKS.
                // Issuer, audience, lifetime and signature are still validated for real.
                o.Authority = null;
                o.Configuration = new OpenIdConnectConfiguration { Issuer = TestTokens.Issuer };
                o.Configuration.SigningKeys.Add(TestTokens.Key);
                o.MapInboundClaims = MapInboundClaims;
            }));
}
```

`ConfigureTestServices` runs after the app's own registration. Setting `Configuration` directly skips the discovery request, and `Authority = null` keeps JwtBearer from fetching one. The same generic factory serves the API and the gateway; `MapInboundClaims` is exposed so one test can turn the default mapping back on and watch the owner receive 403.

9.3 · The expected status of every caller

C# — samples/L11.SecureQuoteApi.Tests/StatusMatrixTests.cs · #region matrix

```
//                          GET quote  POST quotes  accept  referrals
private static readonly (string Caller, int[] Expected)[] Matrix =
[
    ("anonymous",           [401,      401,         401,    401]),
    ("expired",             [401,      401,         401,    401]),
    ("wrong-audience",      [401,      401,         401,    401]),
    ("owner",               [200,      201,         200,    403]),
    ("other-customer",      [403,      201,         403,    403]),
    ("owner-read-only",     [200,      403,         403,    403]),
    ("underwriter",         [200,      403,         403,    200]),
    ("underwriter-rw",      [200,      201,         403,    200]),
    ("back-office-app",     [200,      403,         403,    403]),
];
```

Read it row by row as sentences. The owner may read, create and accept but not list referrals; another customer may create a quote but not read or accept Q-1001; an underwriter who holds the write scope may read, create and list referrals but not accept another customer's quote; the back-office application may read any quote and do nothing else. Heat map 9.8 is parsed from this region at build time, so the picture cannot drift from the test.

9.4 · Commands

Terminal

```
# from the repository root — the SDK pinned by global.json must be installed
dotnet test lesson-11-security-identity/samples/L11.SecureQuoteApi.Tests
dotnet run --project lesson-11-security-identity/samples/L11.TokenLab
dotnet run --project lesson-11-security-identity/samples/L11.ProtectLab
dotnet run --project lesson-11-security-identity/samples/L11.ClaimsVb

# or build, test and run every lesson 11 sample
python 0-script/verify_samples.py --only 11
```

No tenant, no secrets, no network. Restore needs nuget.org once; after that everything runs offline. The two web projects are build-only in `verify_samples.py` — the tests host them in memory.

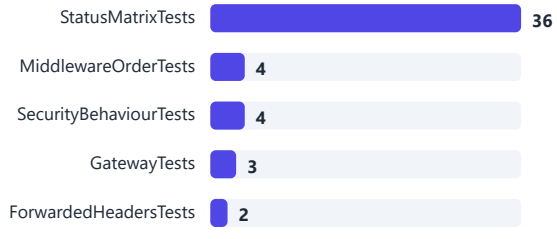
9.5 · What you should see

Output — dotnet test — captured on the build machine

```
Passed!  - Failed:     0, Passed:    49, Skipped:     0, Total:    49, Duration: 1 s
  - L11.SecureQuoteApi.Tests.dll (net10.0)
```

49 passed, 0 failed. The summary is one line, wrapped here before the file name; the duration varies by machine. The console outputs are panels 3.2, 3.6, 7.5 and 8.4. • Measured

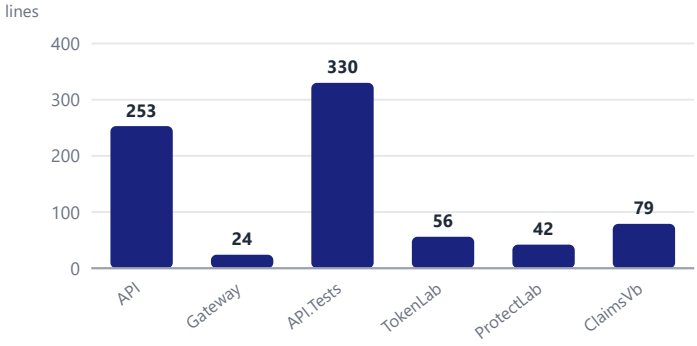
9.6 · Test cases per test class



[Fact] = 1 · [InlineData] = 1 · [MemberData] = matrix rows × endpoints · parsed at build

The matrix is 73% of the suite. The other classes cover middleware order, claim mapping, the gateway's own rejections, throttling, the forwarded-headers trap, an anonymous health check and the Entra registration. • Measured

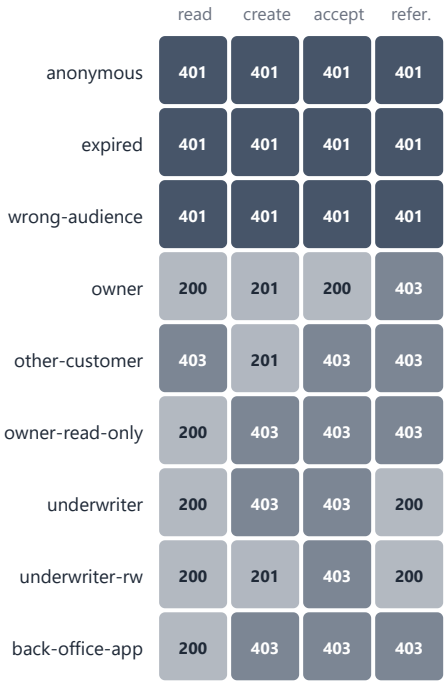
9.7 · Lines of code per project



non-blank, non-comment lines of .cs and .vb per project · measured at build

SecureQuoteApi.Tests is the largest project, at 330 lines. The API (253 lines) and its tests (330) are the same order of size: proving the rules takes about 1.3× the code that writes them. • Measured — a size signal, not a quality score.

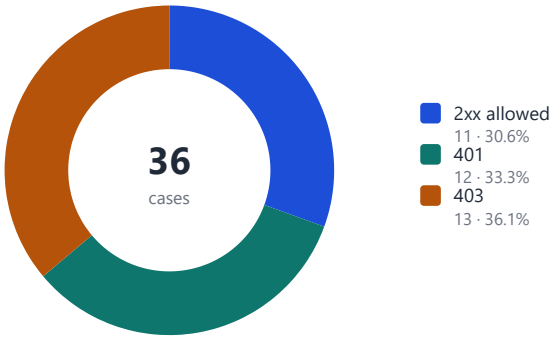
9.8 · The status matrix as a heat map



read = GET quote · create = POST /quotes · accept = POST accept · refer. = GET referrals · parsed from StatusMatrixTests at build

The darker the cell, the earlier the request stopped. The top three rows are uniformly 401: those callers carry no valid identity, so the failed policy ends in a challenge. The 403 cells below them are the authorization design of section 6 — three of them, `other-customer` reading and accepting Q-1001 and `underwriter-rw` accepting it, are decided by the owner handler rather than a policy. • Measured

9.9 · Outcomes across the matrix



count of expected statuses in the matrix region · blue = allowed · teal = 401 · amber = 403

Only 11 of 36 cases are allowed. A security matrix that is mostly green is testing the happy path; this one spends most of its cases on the ways a request must fail. • Measured

⚠ Traps

1. **"My claim is called `scp`."** Not with the default `MapInboundClaims = true`: it is `http://schemas.microsoft.com/identity/claims/scope`, and `roles` is renamed too. Set `MapInboundClaims = false` on every scheme, including the gateway's (3.6).
2. **"An expired token is rejected."** For 5 more minutes it is not. Lower `ClockSkew`, especially at a gateway that forwards what it accepts (3.3).
3. **"Authentication middleware is registered, so the endpoint is protected."** Only endpoints with authorization metadata are; set a fallback policy (2.5). And put `UseAuthentication` before `UseAuthorization` — the reverse compiles and returns 401 (2.4).
4. **"The audience is my `api://` URI."** Entra v2 tokens carry the API's client ID; Cognito access tokens may carry no aud at all (4).
5. **"Each instance can make its own keys."** A Data Protection ring per ECS task means cookies and protected links fail on the next task. Persist and share the ring (7.5).

✓ Checkpoint — you are ready for lesson 12 when

1. `python 0-script/verify_samples.py --only 11` reports 6/6 passed on your machine.
2. You can explain every cell of matrix 9.3 by pointing at the policy, handler or validation step that produces it.
3. Given a Spring Security configuration, you can write the equivalent policies and name the three defaults you must change.
4. You can choose between authorization code + PKCE, on-behalf-of, client credentials and workload identity federation for a new caller, and say which claim the API checks for each.

🔍 Check yourself

1. A request with a valid token gets 401. Name two causes this lesson demonstrated.
2. What is the default `ClockSkew` in .NET, what is Spring Security's, and why does the difference matter more at a gateway?
3. Why does `RequireClaim("scp", "Quotes.Read")` reject a token whose scopes include Quotes.Read?
4. Which claim carries application permissions in an app-only Entra token, and why can that token not be used on-behalf-of?
5. What does a Cognito access token lack that JwtBearer checks by default, and what do you validate instead?
6. Why can the YARP gateway not return the 403 for another customer reading Q-1001, and what does checking that the quote exists before checking its owner reveal to a caller?
7. Why do an API's instances need one Data Protection key ring, and what happens without it?

Lesson 11 in one line: ASP.NET Core authenticates with schemes and authorizes with named policies in a pipeline whose order you write; set `MapInboundClaims = false`, a short `ClockSkew` and a fallback policy, check one resource inside the endpoint, prefer federated credentials to secrets, and prove it all with test-signed tokens.
Next: lesson 12 (Cloud-Native .NET on AWS & Modernization) — deploying this API to AWS Lambda and containers, and modernising the Windows / IIS estates it replaces.